Quick Start

The examples in this guide are for demonstration purposes and may not reflect production best practices.

This guide was last updated for UShare v2.0.0. UShare supports Android 6+ (API Level 23) and iOS 14+.

Breaking Changes Notice

UShare v2.0.0 introduces a new API centered around `ShareSheetManagerV2`.

The previous `ShareSheetManager` has been sealed and deprecated. Existing applications will continue to work without modification, but new projects should use `ShareSheetManagerV2`.

The new API adds:

- Explicit share event identifiers.
- Result callbacks through `OnResult`.
- Dedicated iOS image-sharing APIs that avoid temporary files.
- Access to any temporary files created during share operations.

Unlike previous versions, temporary files are **no longer deleted automatically**. If you create temporary files through the convenience APIs, your application is responsible for cleaning them up when appropriate.

`ShareSheetManagerV2`

`ShareSheetManagerV2` is the main entry point for the native share sheets.

Access it through its singleton instance:

```
ShareSheetManagerV2.Instance
```

Sharing Text

Use `TryShareText()` to share text:

```
using Uralstech.UShare;

public void ShareText()
{
    ShareSheetManagerV2.Instance.TryShareText(
        "Text",
```

```
        title: "Share it with your friends!"
    );
}
```

The `title` parameter is optional.

- Android: Overrides the chooser title on Android 10+.
- iOS: Passed to the native share sheet where supported.

Receiving Share Results

Each share operation is assigned an event ID, which is returned through `OnResult` when the share activity completes.

Subscribe to `OnResult` to receive these callbacks:

```
using Uralstech.UShare;
using UnityEngine;

public class ShareExample : MonoBehaviour
{
    private void Awake()
    {
        ShareSheetManagerV2.Instance.OnResult.AddListener(OnShareResult);
    }

    private void OnDestroy()
    {
        ShareSheetManagerV2.Instance.OnResult.RemoveListener(OnShareResult);
    }

    private void OnShareResult(int eventId)
    {
        Debug.Log($"Share event completed: {eventId}");
    }
}
```

Completion differs by platform:

- **iOS:** Invoked when the user completes or dismisses the share sheet.
- **Android:** Invoked only when the system reports that a target app was selected. Some share targets do not reliably provide this callback.

Most sharing APIs also provide overloads that accept an explicit `int id` parameter. Overloads without an `id` generate one automatically, while explicit IDs can be useful when

you need to correlate callbacks with specific share requests.

Sharing Binary Data

Use `TryShareBytes()` to share binary data:

```
using Uralstech.UShare;

public void ShareImage()
{
    Texture2D image = new(512, 512);
    Color32[] colors = new Color32[512 * 512];

    for (int i = colors.Length - 1; i >= 0; i--)
        colors[i] = Random.ColorHSV();

    image.SetPixels32(colors);

    ShareSheetManagerV2.EventStatus status =
        ShareSheetManagerV2.Instance.TryShareBytes(
            image.EncodeToJPG(),
            "test.jpg",
            CommonMimeTypes.ImageJpg
        );

    Debug.Log($"Share launched: {status.Success}");
}
```

`TryShareBytes()` writes the data to a temporary file before presenting the system share sheet.

The returned `ShareSheetManagerV2.EventStatus` contains:

- `Success` — Whether the share sheet was presented successfully.
- `CreatedFiles` — Any temporary files created for the operation.

```
ShareSheetManagerV2.EventStatus status =
    ShareSheetManagerV2.Instance.TryShareBytes(
        data,
        "report.pdf",
        CommonMimeTypes.ApplicationPdf
    );

foreach (string path in status.CreatedFiles)
{
```

```
    Debug.Log(path);
}
```

Temporary File Cleanup

Convenience methods such as `TryShareBytes()`, `TryShareImage()`, and `TryShareImages()` may create temporary files.

These files are **not automatically deleted**.

Do not assume they can be removed immediately after the share method returns or `OnResult` fires. Instead, keep the paths returned in `EventStatus.CreatedFiles` and clean them up later—for example:

- During a future application launch.
- As part of periodic cache cleanup.
- When you know they are no longer needed.

Temporary files are created under:

```
ShareSheetManagerV2.Instance.GetDefaultBasePath();
```

Since this location is intended for cached data, cleanup is optional, though periodically removing stale files can reduce storage usage.

Sharing Images

`TryShareImage()` behaves similarly to `TryShareBytes()`, but uses an iOS optimization.

- Android: Writes image data to temporary files.
- iOS: Passes image data directly to the native plugin without creating temporary files.

```
ShareSheetManagerV2.Instance.TryShareImage(
    image.EncodeToPNG(),
    "image.png",
    CommonMimeTypes.ImagePng
);
```

You can also share multiple images with `TryShareImages()`:

```
ShareSheetManagerV2.Instance.TryShareImages(
    images,
    fileNames,
```

```
    CommonMimeTypes.ImagePng
);
```

Sharing Existing Files

Share an existing file with `TryShareFile()`:

```
ShareSheetManagerV2.Instance.TryShareFile(
    filePath,
    CommonMimeTypes.ApplicationPdf
);
```

Or multiple files with `TryShareFiles()`:

```
ShareSheetManagerV2.Instance.TryShareFiles(
    filePaths,
    CommonMimeTypes.ApplicationPdf
);
```

How Sharing Files Works on Android

Android apps cannot expose arbitrary internal files directly to other apps.

Instead, Android uses `FileProvider`, which generates content URIs and grants temporary access to the underlying files.

For example, instead of:

```
/data/user/0/com.example.app/cache/ShareCache/image.jpg
```

UShare generates a URI such as:

```
content://com.example.app.FileProvider/shared_data/image.jpg
```

The receiving app is temporarily granted permission to read that file.

Default Configuration

By default, UShare patches your `AndroidManifest.xml` with a `FileProvider` similar to:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
```

```
    <application>
        ...

        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="com.yourcompany.yourapp.FileProvider"
            android:grantUriPermissions="true"
            android:exported="false">

            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_provider_paths" />
        </provider>
    </application>

    ...
</manifest>
```

The default authority is:

```
{Application.identifier}.FileProvider
```

You can override it per request with `ShareOptions`:

```
ShareOptions options = new()
{
    Authority = "com.example.CustomProvider"
};

ShareSheetManagerV2.Instance.TryShareFile(
    path,
    CommonMimeTypes.TextPlain,
    options
);
```

Shareable Folders

On Android, `file_provider_paths.xml` defines which directories can be shared.

By default, UShare generates:

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
    <cache-path
```

```
        name="shared_data"
        path="ShareCache/" />
</paths>
```

This exposes `{cacheDir}/ShareCache/`. For example:

```
{cacheDir}/ShareCache/image.jpg
```

may become:

```
content://com.yourcompany.yourapp.FileProvider/shared_data/image.jpg
```

The default directory can be retrieved using `GetDefaultBasePath()`:

```
ShareSheetManagerV2.Instance.GetDefaultBasePath();
```

`TryShareFile()` and `TryShareFiles()` can only share files exposed through the configured `FileProvider`.

To share files from other locations, either override `file_provider_paths.xml` in **Project Settings → UShare Settings → Custom FileProvider Paths Config**, or provide your own authority through `ShareOptions.Authority`.

If you remove `{cacheDir}/ShareCache` from the exposed paths, `TryShareBytes()`, `TryShareImage()`, and `TryShareImages()` will no longer work. On iOS, you can use `TryShareImageIOS()` and `TryShareImagesIOS()` instead.

Share Options

Additional configuration can be provided through `ShareOptions`:

```
ShareOptions options = new()
{
    Title = "Share",
    Text = "Check this out!"
};
```

Available options:

- `Title` — Optional chooser title. Requires Android 10+ and may not be respected when sharing media.

- `Text` — Additional text to include with shared content.
- `Authority` — Overrides the Android `FileProvider` authority for the request.